

JAVA INTERVIEW PREPARATION

CHAPTER 15

JAVA PLATFORM MODULE SYSTEM

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Java Platform Module System: Reliable Configuration and Strong Encapsulation

Senior / Lead Java Developer Reading Chapter

The Java Platform Module System, introduced in Java 9, adds a level above packages. A module is a named set of packages with an explicit dependency graph and an explicit public surface. JPMS was created to improve reliable configuration, encapsulate JDK internals, and make large applications and the JDK itself easier to structure.

Modules do not replace packages, Maven, or Gradle. Packages organize types and control Java language access. Build tools resolve artifacts and construct builds. JPMS describes which named modules read other modules and which packages they expose at compile time and runtime.

Classpath versus Module Path

On the classpath, JARs contribute classes to a mostly flat namespace. Dependencies may be missing until a code path loads a class, duplicate packages can appear, and every public type is broadly accessible.

On the module path, each named module has a descriptor:

```
module com.acme.orders {  
    requires com.acme.customers;  
    exports com.acme.orders.api;  
}
```

```
com.acme.orders  
|  
+-- reads -----> com.acme.customers  
|  
+-- exports -----> com.acme.orders.api  
|  
+-- concealed --> com.acme.orders.internal
```

The runtime can validate the module graph before normal application execution. This is reliable configuration: required named modules must be present and the graph must be consistent.

module-info.java

The descriptor is normally stored at the source root as `module-info.java` and compiled into `module-info.class`.

```
module com.acme.billing {  
    requires java.sql;  
    requires com.acme.customers;  
  
    exports com.acme.billing.api;  
}
```

A module name should be globally stable, commonly using reverse-domain naming. It is not required to match a Maven group and artifact, but alignment reduces confusion.

The descriptor is architecture as code. It reveals direct dependencies and intentional public packages without inspecting build files and implementation imports.

requires and Readability

`requires module.name` adds a dependency. The requiring module reads the required module, which allows access to its exported packages.

```
module com.acme.reports {  
    requires com.acme.orders;  
}
```

Readability is not the same as Java visibility. Access requires both:

```
module A reads module B  
      +  
module B exports package p  
      +  
target type/member is accessible  
      =  
legal access from A
```

If any condition fails, compilation or runtime access fails.

exports: Public API by Package

Public classes in a non-exported package are public only inside their module boundary.

```
module com.acme.payments {  
    exports com.acme.payments.api;  
}
```

`com.acme.payments.internal` remains strongly encapsulated even if its classes are declared public for use across packages inside the module.

Qualified exports expose a package only to named friends:

```
exports com.acme.payments.testing to com.acme.payments.tests;
```

Use qualified exports sparingly. Many friend relationships can create tightly coupled modules and unclear public boundaries.

opens: Reflection Is Different from Compilation

Frameworks often use deep reflection to access non-public fields, constructors, or methods. Exporting a package does not automatically permit deep reflection.

```
module com.acme.customerapp {  
    opens com.acme.customerapp.persistence to org.hibernate.orm.core;  
    opens com.acme.customerapp.web to com.fasterxml.jackson.databind;  
}
```

`opens` permits runtime reflective access but does not make the package a normal compiled API. `open module` opens every package:

```
open module com.acme.legacyapp {  
    requires com.fasterxml.jackson.databind;  
}
```

An open module eases migration but sacrifices much strong encapsulation. Prefer targeted `opens` once framework requirements are understood.

requires transitive

Use `requires transitive` when consumers of your exported API must also read another module because that module's types appear in your public signatures.

```
module com.acme.orders.api {  
    requires transitive com.acme.money;  
    exports com.acme.orders.api;  
}
```

If `Order.total()` returns `com.acme.money.Money`, a consumer of the order API needs readability to the money module.

Do not mark every dependency transitive. It expands the effective API and couples downstream modules to implementation choices.

```
consumer --> orders.api --transitive--> money  
      |  
      +----- reads money automatically
```

requires static

`requires static` expresses a dependency required at compile time but optional at runtime.

```
module com.acme.annotations {  
    requires static org.jetbrains.annotations;  
}
```

This is useful for source-retention annotations or optional compile-time integration. It does not make arbitrary runtime code safe when the module is absent. Code paths referencing missing runtime types can still fail.

Services: uses and provides

JPMS integrates with `ServiceLoader` for decoupled providers.

```
module com.acme.reports.api {  
    exports com.acme.reports.spi;  
    uses com.acme.reports.spi.ReportExporter;  
}
```

```
module com.acme.pdfexporter {  
    requires com.acme.reports.api;  
    provides com.acme.reports.spi.ReportExporter  
        with com.acme.pdfexporter.PdfReportExporter;  
}
```

The consumer loads providers:

```
ServiceLoader<ReportExporter> exporters =  
    ServiceLoader.load(ReportExporter.class);
```

```
reports application --uses--> ReportExporter SPI  
                        ^  
                        |  
pdf module --provides implementation
```

Services avoid direct provider dependencies, but still need selection, duplicate-provider policy, configuration, error handling, and observability.

Unnamed and Automatic Modules

Classpath code belongs to the unnamed module. It reads all named modules and can access their exported packages, but named modules do not automatically read the unnamed module.

A non-modular JAR placed on the module path becomes an automatic module. Its name may come from `Automatic-Module-Name` in the manifest or be derived from the JAR filename. It exports all packages and reads all other modules.

Automatic modules are migration bridges, not ideal final architecture. Filename-derived names can change with artifact naming, and broad exports conceal intended API boundaries. Library publishers can add a stable `Automatic-Module-Name` before shipping a full descriptor.

Split Packages

JPMS does not allow the same package to be supplied by multiple resolved named modules.

```
module A: com.acme.shared.util
module B: com.acme.shared.util -> split package conflict
```

Split packages were possible on the classpath and often appear in legacy systems, generated code, or test utilities. Modularization may require moving classes, merging artifacts, or redesigning ownership.

This constraint clarifies which module owns a package and prevents ambiguous class loading.

Strong Encapsulation and JDK Internals

Code historically accessed internal JDK packages such as `sun.*` through reflection. Modern Java strongly encapsulates most internals.

The correct fix is to use supported public APIs or upgrade the dependent library. Flags such as `--add-exports` and `--add-opens` can bridge migration:

```
--add-opens java.base/java.lang=com.acme.legacy
```

Treat these flags as visible technical debt. They depend on internal structures, weaken encapsulation, and may fail on future JDK versions.

Use `jdeps` to analyze dependencies and internal API use.

Build Tools and Multi-Module Projects

Maven and Gradle can compile with the module path, but their project module and JPMS module are separate concepts. One build project may produce one JPMS module; aggregators may produce none.

Important build concerns include:

- Correct placement of `module-info.java`.
- Test modules and reflective access.
- Annotation processors.
- Generated sources.
- Mixed classpath and module-path dependencies.
- Surefire, test engines, and IDE configuration.
- Packaging runtime images.

Do not modularize only in the IDE. CI, tests, packaging, and production launch commands must exercise the same module graph.

Testing Modular Code

Tests often need access to non-exported packages. Options include testing through exported APIs, a dedicated test-support module, patching a module for tests, or controlled `--add-opens` and `--add-reads` configuration.

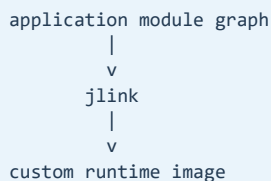
White-box access is sometimes necessary, but indiscriminate test exports can turn internals into accidental application APIs.

Integration tests should launch the real module path and verify:

- Graph resolution.
- Service discovery.
- Serializer and ORM reflection.
- Resource loading.
- Native or agent instrumentation.
- Packaging and startup scripts.

jlink and Custom Runtime Images

`jlink` builds a runtime image containing selected JDK modules and application modules.



Benefits can include smaller distribution size, controlled runtime contents, and no dependency on a separately installed JRE. Operational tradeoffs include image creation in the build pipeline, platform-specific images, security patch rebuilds, container layering, and diagnostic-tool selection.

Custom images are deployment artifacts, not a substitute for minimizing application dependencies.

Migration Strategy

A big-bang conversion is risky. A practical progression is:

1. Upgrade dependencies and eliminate unsupported JDK internals.
2. Resolve split packages and unstable reflection.
3. Add stable Automatic-Module-Name to published libraries where useful.
4. Modularize well-bounded internal libraries.
5. Add targeted opens for frameworks.
6. Modularize application layers and launch on the module path.
7. Consider `jlink` after the graph is stable.

The classpath and module path can coexist during migration, but understand the readability boundary between named and unnamed modules.

Architecture and Module Granularity

A JPMS module should represent a meaningful ownership and dependency boundary. Creating one module per package causes descriptor noise and complex graphs. Putting the entire application in one open module provides little encapsulation.

Good boundaries often align with stable APIs, deployment-independent libraries, or major capabilities. Avoid cyclic dependencies; they indicate responsibilities that may need merging or inversion through an API/SPI module.

```
orders.api <---- orders.impl
  ^              |
  |              v
consumers      payment.api

dependency direction follows stable contracts
```

JPMS enforces declared dependencies but does not create a good architecture automatically.

Resolution Builds the Runtime Graph

The module path is a set of observable modules, not a command to load all of them. Resolution starts from root modules, follows `requires` directives, adds service-provider modules when service binding applies, and produces a configuration graph. If a required named module cannot be found, startup fails before ordinary application logic proceeds.

```
observable modules on module path
[orders] [payments] [postgres.driver] [unused.tool]
  |       |
  +--- root: orders
        |
        requires payments
        |
        resolved configuration
        [orders] -> [payments] -> [java.base]

unused.tool is observable but not resolved
```

This explains why “the JAR is on the module path” is not enough. The module must participate in the resolved graph, or be added as a root where appropriate. Diagnostic work therefore asks three separate questions: is the module observable, was it resolved, and is it readable from the caller?

Use `java --show-module-resolution` to inspect resolution during launch. `java --list-modules` shows available runtime modules, while `java --describe-module name` describes a particular module. These commands answer different questions and are more reliable than guessing from JAR placement.

Readability, Export, and Reflection Are Separate Gates

The access model is easier to remember as a sequence of gates:

```
ordinary compiled access
caller reads target module
+ target exports package
+ Java member is accessible
= legal access

deep reflective access
caller reads target module
+ target opens package to caller
+ reflection request is permitted
= deep access to non-public members
```

An exported package is not automatically open for deep reflection. Conversely, an opened package does not become an ordinary compile-time API. Frameworks that construct objects, inspect private fields, or generate proxies may require `opens`, often targeted to a specific framework module:

```
module com.acme.orders {
    exports com.acme.orders.api;
    opens com.acme.orders.persistence to org.hibernate.orm.core;
}
```

Targeted opening documents the reflective relationship without exposing every package. If a migration depends on command-line `--add-opens`, record the exact consumer and reason; otherwise a temporary compatibility flag quietly becomes permanent production architecture.

API Leakage and requires transitive

Transitive readability is appropriate when a public API exposes types from another module. Consider an orders API returning `Money` from a money module:

```
// com.acme.orders.api
public interface OrderView {
    Money total();
}
```

Consumers cannot use that method signature coherently unless they can also read the module containing `Money`. The descriptor can express this API dependency:

```
module com.acme.orders {
    requires transitive com.acme.money;
    exports com.acme.orders.api;
}
```

Do not apply `transitive` simply because many modules happen to use a library. That widens the architectural contract and couples consumers to a dependency that may be implementation detail. A useful design test is: does an exported signature require consumers to understand the dependency's types?

Services, Providers, and Selection Policy

JPMS integrates with `ServiceLoader`, allowing a consumer to depend on a service contract without reading provider implementation packages directly.

consumer module		provider modules
uses PaymentProvider	+-->	StripePaymentProvider
+-- ServiceLoader	+-->	TestPaymentProvider

```
module com.acme.checkout {
    requires com.acme.payment.api;
    uses com.acme.payment.api.PaymentProvider;
}

module com.acme.payment.stripe {
    requires com.acme.payment.api;
    provides com.acme.payment.api.PaymentProvider
        with com.acme.payment.stripe.StripePaymentProvider;
}
```

Discovery is only one part of plugin design. The application still needs policy for zero providers, multiple providers, ordering, configuration, health checks, provider failure, and security. `ServiceLoader` removes hard-coded construction; it does not create a complete extension platform.

ModuleLayer and Plugin Isolation

The boot layer contains the modules used to start the application. Additional `ModuleLayer` instances can create separate configurations and class-loader relationships for advanced plugin or container scenarios. A layer is not merely a namespace; it participates in class identity. The same class name loaded by different class loaders represents different runtime types.

Use layers only when dynamic isolation is a real requirement. They complicate service discovery, context propagation, unloading expectations, and diagnostics. Most business applications need a well-designed boot module graph, not a custom plugin container.

Modular Testing and Framework Friction

Tests often require access that production code intentionally forbids. A test framework may need to reflect into a concealed package, and white-box tests may compile as though they were part of the module. Build tools can patch test classes into a module or provide targeted opens for the test runtime.

The important architectural decision is whether a test is exercising public behavior or internal structure. Do not permanently export implementation packages merely to make unit tests convenient. Prefer black-box tests through exported APIs, and keep any test-only opens or patching visible in build configuration.

Framework integration must be exercised on the real module path. A test that succeeds on the classpath does not prove that reflection, proxies, resources, service providers, serializers, and launch scripts work under JPMS rules.

When JPMS Is Worth the Cost

JPMS is valuable when strong boundaries, explicit runtime configuration, custom images, or a large codebase's dependency discipline justify the migration effort. It may provide limited return for a small deployable service whose artifact boundaries are already controlled and whose framework ecosystem still requires broad reflective access.

This is not a reason to dismiss modules. It is a reminder to state the objective: preventing accidental internal coupling, securing reflection boundaries, validating the runtime graph, or constructing a smaller runtime. Adopt JPMS for an enforceable benefit, not because the JDK itself is modular.

A Strong Interview Explanation

Describe JPMS as a runtime and compile-time module graph above packages, not as a replacement for Maven or Gradle. Explain the three core ideas: `requires` establishes readability, `exports` defines compiled public packages, and `opens` permits deep reflection. Then distinguish the classpath's broad unnamed module from named modules on the module path.

A senior answer should add the operational consequences: resolution can fail early, split packages are prohibited across resolved named modules, reflective frameworks may need targeted opens, automatic modules are migration bridges, and `jdeps` helps expose hidden dependencies. Finish with the trade-off: JPMS can enforce architecture and enable `jlink`, but module boundaries and granularity still require design judgment.

Production Perspective

Module failures commonly appear during migration or framework startup. A serializer cannot reflect into a DTO package. A service provider is present but not declared with `provides`. A dependency becomes an automatic module with a changed filename-derived name. Tests pass on the classpath while the production module path fails. A command line accumulates broad `--add-opens` flags that hide obsolete libraries.

When diagnosing:

- Inspect the resolved module graph and launch mode.
- Identify which module reads which target.
- Check whether the package is exported or opened for the required operation.
- Verify automatic module names and duplicate packages.
- Use `jdeps` for missing and internal dependencies.
- Check service `uses/provides` declarations.
- Reproduce with production packaging and flags.

Interview-Ready Summary

JPMS adds named modules above packages. A descriptor declares dependencies with `requires`, compiled API packages with `exports`, reflective access with `opens`, and service relationships with `uses` and `provides`.

Access requires module readability, an exported package, and Java member accessibility. `requires transitive` exposes a dependency to downstream consumers when it forms part of the public API; `requires static` represents a compile-time dependency that may be absent at runtime.

The unnamed module represents classpath code. Automatic modules bridge non-modular JARs onto the module path but broadly export and read, so they are migration tools rather than precise boundaries. Split packages are prohibited across resolved named modules.

Modules improve reliable configuration and strong encapsulation, but framework reflection, testing, build tools, services, and deployment require deliberate configuration. `--add-opens` and related flags are migration escape hatches, not architectural solutions.

Revision Notes

- JPMS was introduced in Java 9.
- A module is a named set of packages with an explicit `module-info.java` descriptor.
- The module path enables graph validation and reliable configuration.
- `requires` establishes readability; access also needs export and normal Java accessibility.
- `exports` defines compiled API; public types in concealed packages remain internal.
- Qualified exports expose packages only to named friend modules.
- `opens` permits deep reflection; prefer targeted opens over an open module.
- `requires transitive` is for dependencies exposed through public API, not implementation convenience.
- `requires static` describes compile-time optional dependencies.
- `uses` and `provides` integrate `ServiceLoader`, but selection and failure policy remain application concerns.
- Classpath code belongs to the unnamed module.
- Automatic modules bridge non-modular JARs; prefer stable names and treat them as migration tools.
- Named modules cannot contain split packages across the resolved graph.
- Replace unsupported JDK internals and treat `--add-opens` or `--add-exports` as migration debt.
- Use `jdeps` to analyze dependencies and internal APIs.
- Maven/Gradle modules and JPMS modules are different concepts.
- Test reflection, services, packaging, and launch scripts on the real module path.
- `jlink` creates custom runtime images that must be rebuilt for platform and security updates.
- Migrate incrementally by fixing internals, splits, names, reflection, and graph boundaries.
- Choose meaningful granularity, avoid cyclic graphs, and remember JPMS cannot design boundaries for you.
- A module may be observable without being resolved; diagnose with `--show-module-resolution`.